

Lesson 60 — Final Capstone Project

🔑 Orbit — A Team Work & Project Management SaaS Backend

The graduation project for the entire course. Design and build the backend for **Orbit**, a multi-tenant, real-time work-management platform in the spirit of Linear / Asana / Jira — the kind of SaaS product a professional backend team ships and operates in production.

This is an **architecture, planning, and requirements** document — the brief a senior backend architect hands a developer. It contains **no implementation code**. Your job is to build it, applying concepts from **every phase (Lessons 1–59)** where they naturally fit.

0. How to Use This Capstone

- **Scope:** A single motivated developer can build the **core** (Milestones 1–6) as a portfolio-grade backend; Milestones 7–8 and the Stretch Features are the "extra mile."
- **Standard:** Production-shaped. Layered architecture, real database with migrations, real auth, real-time, tested, containerized, deployable.
- **The promise:** Orbit is deliberately designed so that **every major concept in the syllabus appears where it genuinely belongs** — not bolted on. Section 3 (Syllabus Coverage Map) is the contract: if you build Orbit, you exercise the whole course.
- **Grade yourself** against the Evaluation Criteria (Section 15).

1. Project Overview

1.1 The product

Orbit is a **B2B SaaS** platform where companies organize their work. A company signs up as an **Organization** (a tenant), invites teammates, and structures work into **Workspaces** → **Projects** → **Tasks**. Teams collaborate in **real time** (live task boards, presence, comments), get **notified** of what matters, track time, attach files, and use an **AI assistant** to summarize and create work from natural language. Admins manage members, roles, billing tiers, integrations (API keys + webhooks), and audit trails.

1.2 The problem it solves

Real teams need one system to (a) capture work, (b) assign and prioritize it, (c) see progress live, (d) be notified without drowning, (e) integrate with other tools, and (f) do it securely with per-role access — all isolated per company. Orbit is that backend.

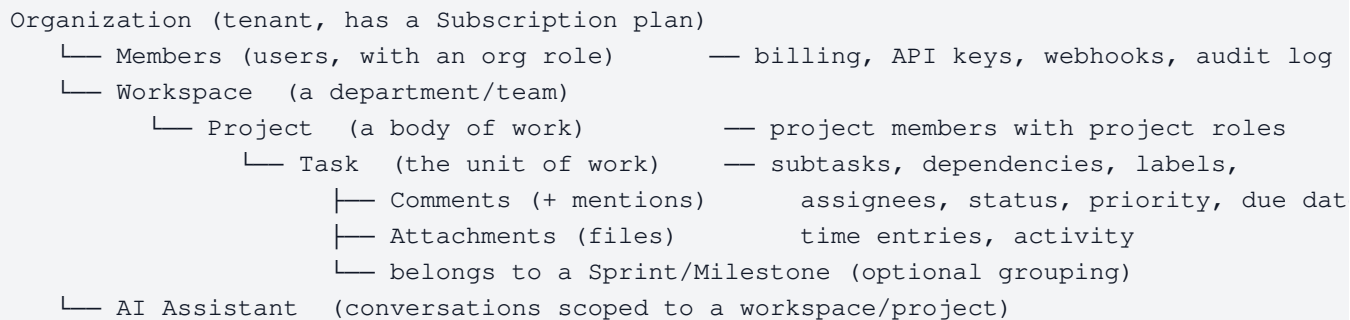
1.3 Who uses it

- **End users** (org members) — create/assign/track work, collaborate live.
- **Org admins / owners** — manage the org, members, roles, billing, integrations.
- **External systems** — CI tools, chatbots, and internal services that call Orbit via **API keys** and receive **webhooks**.
- **Platform operators** (you) — run, monitor, and scale it in production.

1.4 Why it's the right capstone

Orbit is **inherently multi-concern**: multi-tenancy and RBAC (auth), rich relational data (database), live boards (WebSockets), activity streams and AI (SSE/streaming), notifications and digests (background tasks + events), dashboards (caching + aggregation), integrations (API keys, webhooks, rate limits), and a full production/testing/deploy story. It cannot be built as "a pile of CRUD endpoints" — it forces real architecture.

2. Core Domain Model (Mental Model)



Everything is **scoped to an organization** — strict tenant isolation is the platform's #1 invariant. A user in Org A can never see Org B's data.

3. Syllabus Coverage Map — the Contract

This capstone is designed so each concept has a **natural** home. Build these features and you've applied the whole course.

Phase / Lesson	Concept	Where it lives in Orbit
P1 · 1–8	REST design, path/query/body params, <code>Field</code> validation, status codes, <code>HTTPException</code> , <code>Response</code>	Every resource endpoint; task/project/comment CRUD with proper verbs, IDs, and codes
P2 · 9	Pydantic validators (<code>field_</code> / <code>model_validator</code>), <code>model_config</code>	Cross-field rules: due-date after start, priority/status enums, mention parsing, plan-limit checks
P2 · 10–11	<code>response_model</code> , Create/Read/Update schema separation	Hide internal fields (password hashes, raw API keys, soft-delete flags); per-view task shapes

P2 · 12–13	Status codes, custom exception handlers, global error handling	Domain errors → consistent JSON error envelope; 404/403/409/422/429 mapping
P2 · 14	Dependency injection (sub-deps, class-based, <code>yield</code>)	DB session, current user, current org/tenant resolver, pagination params, permission checks
P2 · 15	Middleware	Request-ID + timing + tenant-context middleware; CORS; GZip
P2 · 16	<code>APIRouter</code> , tags, prefixes, nested routers	One router per module (<code>auth</code> , <code>orgs</code> , <code>projects</code> , <code>tasks</code> , ...), versioned under <code>/api/v1</code>
P2 · 17	Form data & file uploads	Task/comment attachments (upload, size/type validation, stored via metadata)
P2 · 18	Headers & cookies	<code>X-Request-ID</code> , <code>X-Org-Id</code> context, <code>Idempotency-Key</code> , refresh-token cookie option
P2 · 19	Static/templates	(Light) rendered email templates for invites/digests; a minimal WS test page
P3 · 20–21	SQLAlchemy 2.0, models, relationships	The full relational schema (Section 6): O2M, M2M, self-referential, association objects
P3 · 22–23	DB session dependency, CRUD, <code>from_attributes</code> , schema/model separation	The repository/service layers; ORM→schema serialization
P3 · 24	Alembic migrations	Every schema change ships as a reversible migration; <code>upgrade head</code> builds the DB
P3 · 25	Async SQLAlchemy	Async engine/session throughout; <code>selectinload</code> for relationships
P3 · 26	<code>SQLModel</code> (<i>alt</i>)	Optional: build one bounded context (e.g. AI) with <code>SQLModel</code> to compare
P3 · 27	Redis (cache) + Mongo (<i>optional</i>)	Redis for cache/rate-limit/pub-sub; optional Mongo for the high-volume activity log
P4 · 28	Async vs sync, <code>run_in_threadpool</code>	Async endpoints; offload blocking work (file hashing, PDF export)
P4 · 29	Auth: bcrypt, OAuth2, JWT (access + refresh), RBAC, API keys	Full auth layer: user login, refresh, org/project RBAC, integration API keys
P4 · 30	Background tasks	Send invite/notification emails, generate digests, process webhooks, thumbnail files
P4 · 31	WebSockets	Live project board: task moves, presence, typing, real-time comments (per-workspace fan-out)

P4 · 32	SSE / streaming	Per-user activity/notification feed stream; AI assistant token streaming
P4 · 33	CORS	Configured for the (hypothetical) web/mobile client origins
P4 · 34	Rate limiting	Per-user + per-API-key + per-plan limits; token budget for the AI assistant
P4 · 35	Caching	Dashboard/report responses, project member lists, permission lookups (with invalidation)
P4 · 36	Pagination	Cursor pagination for task lists / activity feed; offset for admin tables
P4 · 37	Filtering/sorting/searching	Task list: filter by status/assignee/label/due, full-text-ish search, whitelisted sort
P4 · 38	i18n <i>(optional)</i>	Localized notification/email copy negotiated from <code>Accept-Language</code>
P4 · 39	GraphQL <i>(optional)</i>	A read-only GraphQL surface for flexible reporting/ analytics
P5 · 40–43	Testing + coverage + CI	<code>pytest</code> , <code>TestClient/httpx.AsyncClient</code> , isolated test DB, <code>dependency_overrides</code> , coverage gate in CI
P6 · 44	Production project structure	Layered <code>app/</code> (routers/services/repositories/models/ schemas/core/db)
P6 · 45	Config management	<code>pydantic-settings</code> , <code>.env</code> , dev/staging/prod
P6 · 46	Logging	Structured JSON logs + request-ID tracing across a request
P6 · 47	Security best practices	HTTPS/HSTS, secure headers, parameterized queries, secret management, least privilege
P6 · 48	Performance	Fix N+1 (eager loading), connection pooling, indexes, cache hot paths
P6 · 49	Docker	Multi-stage Dockerfile; <code>docker-compose</code> (app + Postgres + Redis)
P6 · 50	Production server	Gunicorn + Uvicorn workers behind Nginx
P6 · 51	Deployment	Deploy to a PaaS / Cloud Run; managed Postgres + Redis
P6 · 52	CI/CD	GitHub Actions: test → build image → migrate → deploy, gated on <code>main</code>

P6 · 53	Monitoring	/health/live + /health/ready, /metrics (Prometheus), Sentry, request tracing
P6 · 54	API versioning	/api/v1; a documented deprecation path for /api/v2
P6 · 55	OpenAPI customization	Rich metadata, examples, tags, hidden internal routes, <code>servers</code> list
Bonus · 56–57	Microservices + events (<i>stretch</i>)	A separate Notification/Webhook worker consuming domain events (<code>task.assigned</code> , ...)
Bonus · 58	gRPC (<i>stretch</i>)	Internal API↔worker calls over gRPC (vs the event bus)
Bonus · 59	LLM/AI patterns	The AI assistant: token streaming, per-user token budgets, conversation state

4. User Roles & RBAC

Orbit has **two role scopes**: organization-level and project-level. A user's effective permission on a resource is the combination.

4.1 Organization roles

Role	Responsibilities	Can	Cannot
Owner	Owens the tenant	Everything below + transfer/delete org, manage billing	(nothing restricted)
Admin	Runs the org day-to-day	Manage members/roles, workspaces, API keys, webhooks, view audit log	Delete the org, change billing owner
Billing Admin	Finance	View/manage subscription & invoices	Manage members or content
Member	Regular teammate	Create/join projects, create tasks, comment, use AI	Manage org settings, members, billing
Guest	External collaborator	Access only explicitly-shared projects; limited actions	See other projects, org settings, member list

4.2 Project roles

Role	Can
Lead	Manage the project, its members, sprints, settings; all task actions

Contributor	Create/edit/assign/move tasks, comment, attach files, log time
Viewer	Read-only: view tasks, comments, activity; no writes

4.3 Machine identity

- **API keys** (per org, scoped, hashed at rest) authenticate **integrations/services** — not humans. Each key carries a scope set (e.g. `tasks:read`, `tasks:write`, `webhooks:manage`) and its own rate limit.

4.4 RBAC enforcement rules

- Every data-access dependency resolves (**current user**, **current org**, **current project**) and checks membership + role before the handler runs.
- **Tenant isolation is absolute**: cross-org access returns **404** (never 403 — don't leak existence).
- Insufficient role within a visible resource returns **403**.
- Guests are constrained to a per-project allowlist.

5. Functional Requirements by Module

Each module: **what** it does, **why** it exists, **who** uses it, **business rules**, **validations**, **permissions**.

5.1 Identity & Auth

- **What**: Registration, login (OAuth2 password flow → JWT **access + refresh**), token refresh, logout, password change/reset, "me" profile.
- **Why**: Prove identity; everything else depends on it.
- **Who**: Everyone.
- **Rules**: Passwords hashed with bcrypt; access tokens short-lived; refresh tokens long-lived and revocable; email must be verified before joining orgs (optional).
- **Validations**: email format + uniqueness; password length/strength (respect bcrypt 72-byte limit); refresh-token type checks.
- **Permissions**: Public (register/login); authenticated (refresh/me/change-password).

5.2 Organizations & Membership

- **What**: Create org, view/update org settings, invite members (email → tokenized invite), accept invite, list/remove members, change member roles, transfer ownership.
- **Why**: The tenant boundary and team roster.
- **Who**: Owners/admins manage; members view roster.
- **Rules**: Creator becomes Owner; an org must always have ≥ 1 Owner; invites expire; a plan caps max members.
- **Validations**: org name/slug (unique slug); role $\in$ allowed set; invite email format.
- **Permissions**: Owner/Admin for management; Member read; Guest cannot list members.

5.3 Workspaces

- **What:** Group projects by team/department; CRUD; membership (inherit org or explicit).
- **Why:** Organize many projects; scope permissions and real-time channels.
- **Who:** Admins/leads create; members access.
- **Rules:** Workspace belongs to exactly one org; deleting requires empty or archives projects.
- **Validations:** name length; unique within org.

5.4 Projects

- **What:** CRUD; archive; project settings; member management with project roles; project-level views/filters.
- **Why:** A bounded body of work with its own team and board.
- **Rules:** Belongs to one workspace; a lead is required; private projects visible only to members (guests via explicit share).
- **Permissions:** Lead manages; Contributor writes tasks; Viewer reads.

5.5 Tasks (the core)

- **What:** Create, read, update, delete (soft), move (status/board column), assign, prioritize, set due dates, subtasks, dependencies, labels, watchers; bulk actions.
- **Why:** The atomic unit of work — the heart of the product.
- **Who:** Contributors+ within a project.
- **Rules:**
 - Status transitions are **controlled** (`todo` → `in_progress` → `in_review` → `done`; reopen allowed).
 - A task cannot be marked `done` while it has open **blocking dependencies**.
 - Subtasks belong to a parent task in the same project; deleting a parent handles its subtasks per policy.
 - Assignee must be a project member; watchers get notifications.
 - Moving/assigning is **atomic** and emits a domain event + real-time update.
- **Validations:** title 1–200; description length; priority/status enums; due date not absurd; label ids exist; assignee is a member.
- **Permissions:** Contributor+ to write; Viewer read-only; guests per project.

5.6 Comments & Mentions

- **What:** Threaded comments on tasks; `@mention` users; edit/delete own; mention triggers a notification.
- **Rules:** Only author (or lead/admin) edits/deletes; mentions must reference project members.
- **Real-time:** New comments broadcast to the task/project channel.

5.7 Attachments (File Uploads)

- **What:** Upload files to tasks/comments (multipart); list/download/delete; store metadata (never raw paths in responses).
- **Rules:** Allowed types (images, PDF, docs); size limit enforced; virus/large-file handling out of scope but size-guarded; thumbnails generated by a background task.
- **Validations:** content-type + size; owner or lead can delete.

5.8 Sprints / Milestones (grouping)

- **What:** Group tasks into time-boxed sprints or milestones; sprint burndown data.
- **Rules:** A task belongs to at most one active sprint; closing a sprint rolls over unfinished tasks (policy).

5.9 Time Tracking

- **What:** Start/stop or manual time entries per task; per-user and per-project totals; reports.
- **Rules:** Entries owned by the logging user; totals feed dashboards (cached).

5.10 Search, Filter, Sort, Pagination

- **What:** List tasks/projects/activity with `q` search, filters (status, assignee, label, due range, priority), whitelisted sort, and pagination (cursor for feeds/tasks, offset for admin).
- **Rules:** All queries tenant- and permission-scoped; sort fields whitelisted (injection-safe).

5.11 Notifications & Activity Feed

- **What:** Per-user notifications (mention, assignment, due-soon, status change); mark read; preferences; a **live activity feed** (SSE); optional email digests.
- **Why:** Keep users informed without polling.
- **How:** Domain events → notification service (background/worker) → stored notification + real-time push (SSE/WS).
- **Rules:** Respect user preferences and mute settings; digest batched by a scheduled task.

5.12 Real-Time Collaboration

- **What:** WebSocket connection per workspace/project: live task moves, new comments, presence ("who's online"), typing indicators.
- **Rules:** Authenticated before `accept()` (token via query/first message); a Connection Manager per workspace; **Redis Pub/Sub** for multi-worker fan-out.

5.13 AI Assistant (LLM)

- **What:** A conversational assistant scoped to a project/workspace: summarize a project's status, turn natural language into draft tasks, answer "what's overdue?", draft a comment — responses **streamed token-by-token** (SSE).
- **Why:** The modern, high-value SaaS feature; showcases Lesson 59.
- **Rules:** Per-user/plan **token budget** (429 when exhausted); conversation history persisted; the model only "sees" data the user is permitted to (permission-filtered context); cost/usage metered.

5.14 Integrations: API Keys & Webhooks

- **What:** Org admins mint scoped **API keys** for external systems; register outbound **webhooks** for events (`task.created`, `task.assigned`, `task.completed`, ...); webhook deliveries are signed, retried, and dead-lettered.
- **Rules:** Keys hashed at rest, shown once, rotatable; webhook payloads HMAC-signed; delivery via background worker with retry/backoff.

5.15 Billing / Plans

- **What:** Subscription plan per org (Free / Pro / Enterprise) that **caps limits** (members, projects, AI tokens, rate limits) and gates features.
- **Rules:** Exceeding a plan limit returns a clear 403/409 with an upgrade hint; billing changes are audited. (Payment-provider integration is out of scope — model the plan/limits only.)

5.16 Audit Log

- **What:** Append-only record of security- and data-sensitive actions (role changes, deletions, key mints, logins); queryable by admins.
- **Rules:** Immutable; tenant-scoped; high-volume → candidate for Mongo or a partitioned table.

5.17 Admin & Health

- **What:** Internal health/readiness endpoints, metrics, and (hidden-from-public-docs) operational routes.

6. Database Design

PostgreSQL, async SQLAlchemy, Alembic migrations. Every table is **org-scoped** (directly or transitively). Below: entities, key columns, relationships, and rationale.

6.1 Entities & relationships

Table	Key columns	Relationships / notes
users	id, email (unique), hashed_password, full_name, is_active, created_at	Global accounts (a user can belong to many orgs)
organizations	id, name, slug (unique), owner_id → users, plan_id → plans, created_at	The tenant root
plans	id, name (free/pro/enterprise), max_members, max_projects, max_ai_tokens_daily, rate_limit_rpm	Drives limits (Section 5.15)
organization_members	id, org_id → orgs, user_id → users, role, joined_at, unique(org_id, user_id)	Association object (user↔org M2M with a role)
invitations	id, org_id → orgs, email, token (unique), role, invited_by → users, status, expires_at	Tokenized invites
workspaces	id, org_id → orgs, name, created_by → users	O2M from org
projects	id, workspace_id → workspaces, key (short code), name, status, is_private, lead_id → users, created_at	O2M from workspace
project_members	id, project_id → projects, user_id → users, role, unique(project_id, user_id)	Association object (project role)

tasks	id, project_id → projects, title, description, status, priority, assignee_id → users nullable, sprint_id → sprints nullable, parent_id → tasks nullable (self-ref subtasks), due_at, position, is_deleted, created_by → users, created_at	The core entity
task_dependencies	blocker_task_id → tasks, blocked_task_id → tasks, unique pair	Self-referential M2M (blocking graph)
labels	id, org_id → orgs, name, color, unique(org_id, name)	Reusable tags
task_labels	task_id → tasks, label_id → labels, unique pair	M2M task ↔ label
task_watchers	task_id → tasks, user_id → users, unique pair	M2M (notification subscribers)
comments	id, task_id → tasks, author_id → users, body, parent_id → comments nullable (threads), edited_at, created_at	O2M + self-ref threads
mentions	id, comment_id → comments, mentioned_user_id → users	Drives mention notifications
attachments	id, task_id → tasks nullable, comment_id → comments nullable, uploader_id → users, filename, content_type, size_bytes, storage_key, created_at	Metadata only; file bytes in object storage/local uploads/
sprints	id, project_id → projects, name, starts_on, ends_on, status	Time-boxing
time_entries	id, task_id → tasks, user_id → users, minutes, note, occurred_on	Time tracking
notifications	id, user_id → users, org_id → orgs, type, payload (JSON), is_read, created_at	Per-user inbox
activity_events	id, org_id → orgs, actor_id → users, entity_type, entity_id, verb, metadata (JSON), created_at	Feed + partial audit; high-volume (index heavily / consider Mongo)
api_keys	id, org_id → orgs, name, key_hash, prefix, scopes (JSON), last_used_at, created_by, revoked_at	Machine identity, hashed
webhooks	id, org_id → orgs, url, secret, events (JSON), is_active, created_at	Outbound integrations
webhook_deliveries	id, webhook_id → webhooks, event_type, payload, status, attempts, next_retry_at	Retry/dead-letter tracking
ai_conversations	id, org_id → orgs, project_id → projects nullable, user_id → users, created_at	AI assistant sessions
ai_messages	id, conversation_id → ai_conversations, role, content, input_tokens, output_tokens, created_at	Conversation history + usage

subscriptions/ usage_counters	id, org_id, period, ai_tokens_used, ...	Meter plan usage
audit_log	id, org_id, actor_id, action, target, ip, created_at	Immutable security log

6.2 Relationship summary

- **One-to-many:** org→workspaces→projects→tasks→comments; task→subtasks; project→sprints; user→notifications.
- **Many-to-many (association objects with roles):** user↔org (organization_members), user↔project (project_members).
- **Many-to-many (plain junctions):** task↔label, task↔watcher.
- **Self-referential:** task subtasks (parent_id), comment threads (parent_id), task dependency graph (task_dependencies).

6.3 Constraints & indexes

- **Uniques:** email, org slug, project key per org, membership pairs, label per org, dependency pairs.
- **Foreign keys** with sensible on-delete (cascade for owned children; restrict where it would orphan integrity, e.g. can't delete a user who is the sole org owner).
- **Indexes:** every FK; tasks (project_id, status), tasks (assignee_id), tasks (due_at) for the hot task-list filters; activity_events (org_id, created_at) for the feed; notifications (user_id, is_read); api_keys (prefix) for lookup.
- **Check constraints:** enum-like columns (status/priority/role), non-negative token/minute counters.

7. API Design

Everything under `/api/v1`. JSON. Auth via `Authorization: Bearer <jwt>` (users) or `X-API-Key` (integrations). Consistent error envelope: `{ "error_code": "...", "detail": "...", "request_id": "..." }`. Standard codes: 200/201/204, 400, 401, 403, 404, 409, 422, 429, 5xx.

Representative endpoints grouped by module (not exhaustive; every list endpoint supports pagination + filtering where noted).

Auth

Method	Path	Auth	Notes
POST	<code>/auth/register</code>	public	201; unique email (409 on dup)
POST	<code>/auth/login</code>	public (form)	200 → access + refresh; 401 generic
POST	<code>/auth/refresh</code>	refresh token	new access token; 401 if invalid/expired
POST	<code>/auth/logout</code>	user	revoke refresh token

GET	/auth/me	user	current profile
POST	/auth/change-password	user	re-auth; 422 on weak

Organizations & Members

Method	Path	Auth	Notes
POST	/orgs	user	creator = Owner
GET	/orgs	user	orgs the caller belongs to
GET/PATCH	/orgs/{org_id}	member / admin	tenant-scoped (404 if outside)
POST	/orgs/{org_id}/invites	admin	201; plan member cap (409)
POST	/invites/{token}/accept	user	joins org
GET	/orgs/{org_id}/members	member	paginated; guests blocked
PATCH	/orgs/{org_id}/members/{user_id}	admin	change role; keep ≥1 owner (409)
DELETE	/orgs/{org_id}/members/{user_id}	admin	remove member

Workspaces & Projects

Method	Path	Auth	Notes
POST/GET	/orgs/{org_id}/workspaces	admin / member	
POST/GET	/workspaces/{id}/projects	lead / member	private projects filtered by membership
GET/PATCH/DELETE	/projects/{id}	project role	soft-delete/archive
POST/DELETE	/projects/{id}/members	lead	assign project roles

Tasks

Method	Path	Auth	Notes
POST	/projects/{id}/tasks	contributor+	validations; 201; emits <code>task.created</code>
GET	/projects/{id}/tasks	viewer+	filter/sort/search + cursor pagination
GET/PUT/DELETE	/tasks/{id}	role-based	soft-delete

POST	/tasks/{id}/move	contributor+	status/board move; atomic ; 409 on invalid transition or open blockers
POST	/tasks/{id}/assign	contributor+	assignee must be member (409)
POST/DELETE	/tasks/{id}/labels, /watchers, /dependencies	contributor+	M2M edits
POST/GET	/tasks/{id}/comments	role-based	mentions → notifications
POST/GET/DELETE	/tasks/{id}/attachments	role-based	multipart; type/size validation (415/413)
POST/GET	/tasks/{id}/time-entries	contributor+	

Notifications & Activity (real-time)

Method	Path	Auth	Notes
GET	/notifications	user	cursor paginated; filter unread
POST	/notifications/{id}/read	user	
GET	/notifications/stream	user	SSE live feed
WS	/ws/workspaces/{id}	user (token)	live board: moves, comments, presence, typing

AI Assistant

Method	Path	Auth	Notes
POST	/ai/conversations	member	scoped to project/workspace
GET	/ai/conversations/{id}/stream	member	SSE token streaming ; per-user token budget (429)
GET	/ai/usage	member/admin	tokens consumed vs plan budget

Integrations & Admin

Method	Path	Auth	Notes
POST/GET/DELETE	/orgs/{org_id}/api-keys	admin	key shown once; hashed at rest
POST/GET/DELETE	/orgs/{org_id}/webhooks	admin	signed deliveries
GET	/orgs/{org_id}/audit-log	admin	paginated, filterable
GET	/orgs/{org_id}/dashboard	member	cached aggregate metrics

GET	/health/live, /health/ready	public	liveness / readiness
GET	/metrics	internal	Prometheus

8. Real-Time & Asynchronous Architecture

- **WebSockets (Lesson 31):** one connection per workspace; a **Connection Manager** tracks sockets; **Redis Pub/Sub** fans events out across workers so a task move on worker A reaches clients on worker B.
- **SSE (Lesson 32):** the per-user notification/activity feed and the AI token stream — one-way server→client, auto-reconnecting.
- **Background tasks (Lesson 30):** send invite/notification/digest emails, generate attachment thumbnails, deliver webhooks (with retry/backoff), recompute cached dashboards. Fire-and-forget for loss-tolerant work; note where a real queue (Celery/RQ/ARQ) would be the production choice.
- **Domain events (Bonus 57):** `task.assigned`, `task.completed`, `comment.mentioned`, `member.invited` published to an event bus; the **Notification/Webhook worker** consumes them → decoupled, extensible, at-least-once with idempotent handlers.

9. Production Architecture

Concern	Approach (lesson)
Config	<code>pydantic-settings</code> + <code>.env</code> ; dev/staging/prod; secrets from env/secret manager (45, 47)
Logging	Structured JSON logs, request-ID tracing via middleware (46)
Monitoring	<code>/health/live</code> + <code>/health/ready</code> , <code>/metrics</code> (Prometheus), Sentry error tracking, optional OpenTelemetry tracing (53)
Caching	Redis for dashboards, permission lookups, member lists; TTL + explicit invalidation on writes (35)
Rate limiting	Per-user, per-API-key, per-plan; Redis-backed for multi-worker; AI token budgets (34)
Security	HTTPS/HSTS, secure headers, parameterized queries (ORM), hashed secrets, least-privilege DB user, generic auth errors (47)
Performance	Eager-load to kill N+1, connection pooling, targeted indexes, cache hot paths, cursor pagination on big lists (48)
Docker	Multi-stage image; <code>docker-compose</code> for app + Postgres + Redis (49)
Server	Gunicorn + Uvicorn workers behind Nginx reverse proxy (50)
Deployment	PaaS or Cloud Run + managed Postgres/Redis; migrations run at release (51)
CI/CD	GitHub Actions: lint/test/coverage → build image → <code>alembic upgrade head</code> → deploy, gated on <code>main</code> (52)

Versioning	/api/v1; documented deprecation strategy for a future /api/v2 (54)
Docs	Customized OpenAPI: metadata, examples, tags, hidden internal routes, <code>servers</code> list (55)

10. Folder Structure (Production, Layered)

```

orbit/
├─ app/
│   ├─ main.py                # assembly: app, middleware, routers, exception hand
│   ├─ core/                  # config, security (JWT/bcrypt), logging, events
│   │   ├─ config.py
│   │   ├─ security.py
│   │   ├─ logging.py
│   │   └─ events.py          # event bus / publisher
│   ├─ db/                    # engine, async session, Base, get_db
│   │   └─ base.py
│   ├─ models/                # SQLAlchemy models (one file per aggregate)
│   ├─ schemas/               # Pydantic Create/Read/Update per resource
│   ├─ repositories/          # data-access (queries, no HTTP, no business rules)
│   ├─ services/              # business logic (task workflow, RBAC, billing limit
│   ├─ api/
│   │   ├─ deps.py            # current_user, current_org, permissions, pagination
│   │   └─ v1/
│   │       ├─ routes/        # auth, orgs, workspaces, projects, tasks, comments,
│   │       │               # attachments, notifications, ai, integrations, ad
│   │       └─ ws.py          # websocket endpoints + connection manager
│   ├─ realtime/              # connection manager, redis pub/sub, sse streams
│   ├─ workers/               # background tasks + event consumers (notifications,
│   ├─ exceptions.py          # domain errors + handlers
│   └─ i18n/                  # (optional) translation catalogs
├─ alembic/                   # migrations
│   ├─ env.py
│   └─ versions/
├─ tests/
│   ├─ conftest.py            # fixtures: test DB (StaticPool), client, auth, over
│   ├─ unit/                  # services/repositories in isolation
│   ├─ integration/           # DB-backed
│   └─ api/                   # endpoint + auth + RBAC tests
├─ docker/
│   ├─ Dockerfile             # multi-stage
│   ├─ docker-compose.yml     # app + postgres + redis
│   └─ nginx.conf
├─ .github/workflows/ci-cd.yml
├─ .env.example
├─ pyproject.toml / requirements.txt
├─ alembic.ini
└─ README.md

```

By-layer here for clarity; a large team may migrate to **by-feature** packages (Lesson 44). Keep it

consistent.

11. Testing Strategy

Test kind	What it covers	Tools (lessons 40–43)
Unit	Services/repositories in isolation (task-transition rules, RBAC checks, billing limits, token budgets)	pytest, no HTTP
Integration	Repositories against a real (test) DB; transactions; migrations apply cleanly	isolated test DB (<code>StaticPool</code>), per-test isolation
API	Endpoints end-to-end (create→assign→move→comment), status codes, response shapes	<code>TestClient</code> / <code>httpx.AsyncClient</code>
Auth & RBAC	401/403/404 matrix per role; token refresh; API-key scopes; tenant isolation (Org A can't see Org B)	<code>app.dependency_overrides</code> to inject users/roles
Real-time	WebSocket connect/broadcast/disconnect; SSE token stream framing	<code>TestClient</code> websocket + streaming
External services	AI provider & email mocked via dependency overrides / fakes	mocking
Coverage	Gate in CI (e.g. <code>--cov-fail-under=85</code>), term-missing to find gaps	<code>pytest-cov</code>

Fixtures: a `client`, an isolated `db_session`, factories for orgs/users/projects/tasks, and role-scoped auth headers. Every test independent and order-independent.

12. Development Roadmap (Milestones)

Build incrementally; each milestone yields a working, testable slice.

Milestone 1 — Foundation & Auth

- **Objective:** Runnable layered app; users can register, log in, refresh.
- **Modules:** project structure (44), config (45), logging (46), DB + Alembic (21–24), auth (29), middleware (15), error handling (13).
- **Complexity:** Medium. **Estimated time:** 10–14 h. **Depends on:** —. **Outcome:** authenticated skeleton with migrations, `/health`, structured logs.

Milestone 2 — Tenancy & RBAC

- **Objective:** Orgs, members, invites, workspaces, projects with roles.

- **Modules:** orgs/workspaces/projects, RBAC dependencies, tenant isolation.
- **Complexity:** Medium-High. **Estimated time:** 12–16 h. **Depends on:** M1. **Outcome:** multi-tenant, role-gated resource access; 404-vs-403 discipline.

Milestone 3 — Tasks Core

- **Objective:** The heart — tasks, subtasks, dependencies, labels, comments, mentions, watchers; controlled status transitions.
- **Modules:** tasks, comments, relationships, validators (9), response models (10–11).
- **Complexity:** High. **Estimated time:** 18–24 h. **Depends on:** M2. **Outcome:** full task lifecycle with business rules and atomic moves.

Milestone 4 — Query, Files, Time

- **Objective:** Task list power features + attachments + time tracking.
- **Modules:** filtering/sorting/search (37), pagination (36), file uploads (17), time entries.
- **Complexity:** Medium. **Estimated time:** 10–14 h. **Depends on:** M3. **Outcome:** production-grade list endpoint; uploads; reports data.

Milestone 5 — Real-Time & Notifications

- **Objective:** Live boards + notifications + activity feed.
- **Modules:** WebSockets (31), SSE (32), background tasks (30), events (57), Redis pub/sub.
- **Complexity:** High. **Estimated time:** 16–22 h. **Depends on:** M3. **Outcome:** live collaboration; decoupled notification pipeline.

Milestone 6 — Caching, Rate Limits, Dashboards, Billing

- **Objective:** Performance + cost/limits + aggregate views.
- **Modules:** caching (35), rate limiting (34), performance/N+1 (48), plans/limits, dashboards.
- **Complexity:** Medium-High. **Estimated time:** 12–16 h. **Depends on:** M4–M5. **Outcome:** fast, plan-gated, observable-in-usage app.

Milestone 7 — Integrations & AI

- **Objective:** API keys, signed webhooks, and the streaming AI assistant.
- **Modules:** API keys (29), webhooks (background + retry), AI streaming + token budgets (59).
- **Complexity:** High. **Estimated time:** 16–22 h. **Depends on:** M5–M6. **Outcome:** external integrations + a modern AI feature.

Milestone 8 — Test, Harden, Ship

- **Objective:** Full test suite, monitoring, containerize, CI/CD, deploy.
- **Modules:** testing (40–43), security (47), monitoring (53), Docker (49), server (50), deployment (51), CI/CD (52), versioning (54), OpenAPI (55).
- **Complexity:** High. **Estimated time:** 16–22 h. **Depends on:** all. **Outcome:** a tested, monitored, containerized backend deployed via an automated pipeline.

Time Summary

Estimates assume a developer who has just completed this course (solid intermediate level), simulating the external payment/AI providers rather than fully integrating them.

Milestone	Focus	Hours
M1	Foundation & Auth	10–14
M2	Tenancy & RBAC	12–16
M3	Tasks Core (richest domain)	18–24
M4	Query, Files, Time	10–14
M5	Real-Time & Notifications	16–22
M6	Caching, Rate Limits, Dashboards, Billing	12–16
M7	Integrations & AI	16–22
M8	Test, Harden, Ship	16–22
Core (M1–6)	portfolio-ready	~80–105 h
Full (M1–8)	complete	~110–150 h

On a calendar (full build): ~11–15 weeks part-time (~10 h/week) · ~6–8 weeks at ~20 h/week · ~3–4 weeks full-time.

Adjust for: skill level (junior fresh out of the course $\approx 1.5\text{--}2\times$; experienced backend dev $\approx 0.6\text{--}0.7\times$); wiring a *real* LLM/provider instead of a simulated one (+8–15 h each); and polish level (90%+ coverage, load tests, pixel-perfect OpenAPI can add ~15–25%).

Recommended: build **Core (M1–6)** first — already an interview-worthy, demo-able backend — then decide whether M7–M8 are worth the extra ~30–50 h for your goals. **Core + M8 (ship it)** is the sweet spot for most portfolios.

13. Stretch Features (Beyond the Syllabus)

- **Notification microservice + event bus** — extract notifications/webhooks into a separate service consuming a real broker (RabbitMQ/Kafka) — full Bonus 56–57.
- **gRPC internal API** — API↔worker calls over gRPC with a shared `.proto` (Bonus 58).
- **GraphQL analytics** — a read-only GraphQL surface for flexible reporting (Lesson 39).
- **i18n** — localized notifications/emails via `Accept-Language` (Lesson 38).
- **Full-text search** — Postgres `tsvector` (or Meilisearch/Elasticsearch) for real task search at scale.
- **Saved views & custom fields** — user-defined filters and per-org custom task fields.
- **Optimistic concurrency** — `ETag`/version on tasks to prevent lost updates on concurrent edits.

- **Prompt caching / RAG for the AI assistant** — cache project context; retrieve relevant tasks for grounded answers.
- **Multi-region / read replicas, feature flags, soft-delete + restore + data export (GDPR).**

14. Advanced-Feature Placement (Quick Reference)

Feature	Natural home in Orbit
JWT + refresh tokens	Auth module
API keys	Integrations
RBAC	Org + project permission dependencies
Background tasks	Emails, digests, thumbnails, webhook delivery
WebSockets	Live project board / presence
Streaming (SSE)	Notification feed + AI assistant
Caching	Dashboards, permission/member lookups
Pagination / filter / sort / search	Task lists, activity feed
Rate limiting	Per user/key/plan + AI token budget
File uploads	Task/comment attachments
Async DB + transactions	Everywhere; atomic task moves & billing
Middleware / DI / exception handling	Request context, permissions, error envelope

15. Evaluation Criteria (Grade Yourself)

- **Architecture:** clean layering (routers → services → repositories → models); no business logic in routes; thin `main.py`.
- **Correctness:** RBAC + **tenant isolation** are airtight; task workflow and atomic operations behave under concurrency.
- **Data:** correct relationships (O2M, M2M, self-ref, association objects), constraints, indexes; migrations build from empty and downgrade.
- **Real-time & async:** WS/SSE authenticated and multi-worker-safe (Redis); background/event work decoupled and idempotent.
- **Performance:** no N+1 on hot paths; caching with correct invalidation; bounded, paginated lists.
- **Security:** hashed secrets, parameterized queries, secure headers, least privilege, generic auth errors, no secrets in logs/responses.

- **Testing:** meaningful unit/integration/API/RBAC tests; isolated DB; coverage gate green in CI.
- **Production:** config by environment, structured logs, health + metrics, Dockerized, migrations-at-deploy, CI/CD pipeline, versioned + documented API.

Course Complete

If you build Orbit to this brief, you will have shipped a **multi-tenant, real-time, AI-assisted SaaS backend** — authenticated, role-secured, tested, observable, containerized, and deployed through an automated pipeline. That is the full arc of this course, from your first `@app.get("/")` in Lesson 1 to a production system a company could actually run.

Congratulations — you've reached the end of the FastAPI: Beginner → Advanced course. Now go build it. 🦉

➡ Where to Go From Here

- Build Orbit milestone-by-milestone; treat each milestone as its own PR with tests and a green CI run.
- Revisit any lesson whose concept feels shaky when you reach the milestone that uses it (the Coverage Map in Section 3 is your index).
- Publish it: a deployed Orbit instance + a clear README is a standout portfolio piece for backend interviews and internships.